

Part 1: The Phantom Bug

Deliverable 1: Part 1 — Five Runs

Run	total Hits	$\pi \approx$
1	22,04 5,2 54	1.763 620
2	22,01 7,4 15	1.761 393
3	22,10 5,4 88	1.768 439
4	21,93 2,5 34	1.754 603
5	22,00 7,7 36	1.760 619

Average pi $\approx$ 1.7618 (expected correct value: 3.14159)

Analysis

The expected hit count for 50,000,000 points is $\approx$ 39,270,000 (since $\pi/4 \approx 0.7854$). Instead, every run landed around 22,000,000 — roughly 44% of all increments were silently lost, consistently across all 5 runs.

This is the signature of a data race on `shared.value += 1`: each increment is really a read-modify-write (read the current value, add 1, write it back), and with no lock, multiple processes can read the *same* value at the same moment, then each write back `value+1`, so two (or more) increments collapse into one. The higher the contention — 4 processes hammering the same shared memory location millions of times — the more updates get clobbered.

Part 2: The Synchronization Trap

=== Single-process baseline ===

[Single-process] hits=39271057 pi $\approx$ 3.141685 time=8038ms

=== Multi-process with a Lock around every increment ===

[Locked, 4 procs] hits=39266960 pi $\approx$ 3.141357 time=229121ms

Locked version is 28.51x the single-process time.

Why this happened:

Adding the lock fixed the correctness problem from Part 1 — both versions now land right around 3.1416, so the race is gone. But look at the cost: instead of being *faster* with 4 processes, it's almost 30x slower than doing the whole thing on a single core with no threading at all.

The reason is that I'm locking around a *single increment*, and that happens close to 40 million times. Every single one of those 40 million `shared.value += 1` calls has to:

1. Acquire the lock (this is an OS-level semaphore call since these are separate processes, not threads in one process)
2. Do the tiny write
3. Release the lock

So instead of doing useful work, the 4 processes spend almost all their time taking turns waiting for a lock that protects basically nothing (one integer). And because these are full OS *processes* (not lightweight threads), that locking has to go through inter-process communication, which is way more expensive than a lock inside a single program — that's why my slowdown (28.5x) is even worse than what the Java version usually shows with `synchronized` threads (which are typically just a few times slower, not 30x).

The lesson: correctness came back, but I paid for it with a massive performance hit, because I put the lock in the wrong place — around every single tiny operation instead of around larger chunks of work. This is exactly the setup for Part 3, where the fix is to stop synchronizing on every increment altogether and instead give each process its own private counter, only combining them once at the very end.

Part 3: OpenMP-Style Reduction

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (TN T1)	Efficiency (TSpeedup)
1	22900	1.0x	100%
2	12899	1.78x	88.8%

4	5750	3.98x	99.6%
8	4272	5.36x	67.0%
16	3564	6.43x	40.2%

pi stayed correct (3.1414–3.1416) at every thread count — confirming the reduction pattern (private local counter per process, summed once at the end) fixes both the correctness problem from Part 1 *and* the performance problem from Part 2, without any locking at all.

What the numbers show:

Scaling is close to ideal up through 4 processes — efficiency is 88–99%, meaning I'm getting almost the full benefit of each extra process with barely any overhead. That's the payoff of removing the shared-state lock entirely: with no contention, more workers really do mean proportionally more throughput.

Past 4 processes, efficiency drops off fast: 67% at 8, 40.2% at 16, and only 16.8% at 32 — even though runtime is still technically going down (or, at 32, going back *up* compared to 16). That's the tell that I've run out of real hardware to parallelize onto. My laptop doesn't actually have 32 physical cores; once the process count passes the number of logical cores available, extra processes just get time-sliced onto cores that are already busy, so I get diminishing (and eventually negative) returns rather than continued speedup. (I'd confirm my exact core count with `python -c "import os; print(os.cpu_count())"`, but the shape of this curve — a sharp bend around 4–8 and 32 actually being *slower* than 16 — is a strong sign my physical core count is somewhere around there.)

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

Answer: Going from 8 to 16 processes only improved runtime from 4,272ms to 3,564ms (a modest ~1.2x gain), nowhere near doubling. That's because

there's no such thing as "16 cores" for the OS to hand out here — once the process count exceeds the number of physical cores, extra processes can't get genuinely independent execution; they have to time-share the same cores the first batch was already using. Any gain I still see from 8→16 likely comes from hyperthreading (logical cores sharing one physical core's resources), which helps a little by filling in idle pipeline gaps, but nowhere close to doubling throughput. That's exactly why efficiency drops from 67% to 40.2% — I'm adding processes faster than I'm adding real compute capacity, and by 32 processes it actually backfires (runtime gets worse than at 16) because now I'm paying pure scheduling/context-switching overhead with essentially zero extra hardware to show for it.

2. Why was the synchronized version in Part 2 slower than running on one single core?

Answer: Because the lock in the part2 protected a single tiny operation (`shared.value+=1`) that happened ~39 million times. Every one of those increments had to acquire an OS-level lock, do the write, then release it — and since Part 2 used separate processes (not threads in one program), that locking meant real inter-process synchronization overhead, not just an in-memory lock. With 4 processes constantly fighting over that same lock, almost all their time went to waiting their turn instead of doing useful work, which is exactly why it came out 28.5x slower than the single-process baseline. Part 3 fixes this by never taking a lock in the hot loop at all — each process piles up hits in its own private variable and only reports back once, at the very end, so there's no contention to pay for while the actual work is happening.